

Frontend–Backend Entegrasyonu Teknik Raporu

Backend mimari aşamasından sonra kurulan frontend–backend bağlantısının uçtan uca teknik dökümü: hangi dosya, hangi kod, hangi katmanda, ne işe yarıyor.

Proje: CineSeat — Online Sinema Bilet Rezervasyon Sistemi

Backend: ASP.NET Core Web API · Clean/Onion Architecture · CQRS (MediatR) · PostgreSQL

Frontend: React 19 · Vite · React Query · React Router

Kapsam: Kimlik doğrulama, API sözleşmesi, servis katmanı, hata yönetimi, sayfalama, koltuk kilitleme

Tarih: 28.08.2026

İçindekiler

1. Giriş ve Kapsam
2. Genel Mimari
 - 2.1 Backend katmanları (Onion/Clean Architecture)
 - 2.2 CQRS ve MediatR boru hattı
 - 2.3 Frontend katmanları
 - 2.4 İki uygulama arasındaki genel şema
3. Ağ Katmanı: Portlar, Ortam Değişkenleri, CORS
4. Kimlik Doğrulama (JWT) — Uçtan Uca Akış
5. İstek/Yanıt Sözleşmesi (Contract)
 - 5.1 JSON serileştirme kuralları
 - 5.2 DTO eşleme deseni
6. Merkezi API İstemcisi: `ApiClient.js`
7. Servis Katmanı: Her Dosyanın Görevi
8. CQRS Zinciri — Uçtan Uca Örnek: Giriş (Login)
9. Hata Yönetimi Zinciri
10. Sayfalama Sözleşmesi
11. Özel Akış: Koltuk Kilitleme (Concurrency)
12. Özel Akış: Kampanya/İndirim Önizlemesi
13. Dosya-İşlev Referans Tablosu
14. Canlı Doğrulama Notları
15. Sonuç

1. Giriş ve Kapsam

Bu rapor, CineSeat projesinde backend mimarisinin (Onion/Clean Architecture + CQRS) tamamlanmasının ardından kurulan **frontend-backend entegrasyonunu** teknik düzeyde belgeler. Amaç, iki bağımsız uygulamanın (ayrı süreç, ayrı port, ayrı repo dizini) birbirleriyle nasıl haberleştiğini; hangi dosyanın bu haberleşmede hangi sorumluluğu taşıdığını; kimlik doğrulama, hata yönetimi, veri sözleşmesi ve eşzamanlılık (concurrency) gibi kritik noktaların nasıl çözüldüğünü somut kod örnekleriyle ortaya koymaktır.

Rapor, projeyi hiç görmemiş ama genel web/API kavramlarına hakim bir okuyucunun anlayabileceği ayrıntıda yazılmıştır; aynı zamanda gerçek dosya yolları, gerçek sınıf/fonksiyon adları ve gerçek kod parçaları içerir, böylece teknik bir sunumda doğrudan referans olarak kullanılabilir.

Kapsam dışı: Bu rapor backend'in iç iş mantığını (domain kuralları, veritabanı şeması) veya frontend'in UI/tasarım kararlarını detaylandırmaz; yalnızca **iki tarafın birbirine nasıl bağlandığını** konu alır.

2. Genel Mimari

2.1 Backend Katmanları (Onion / Clean Architecture)

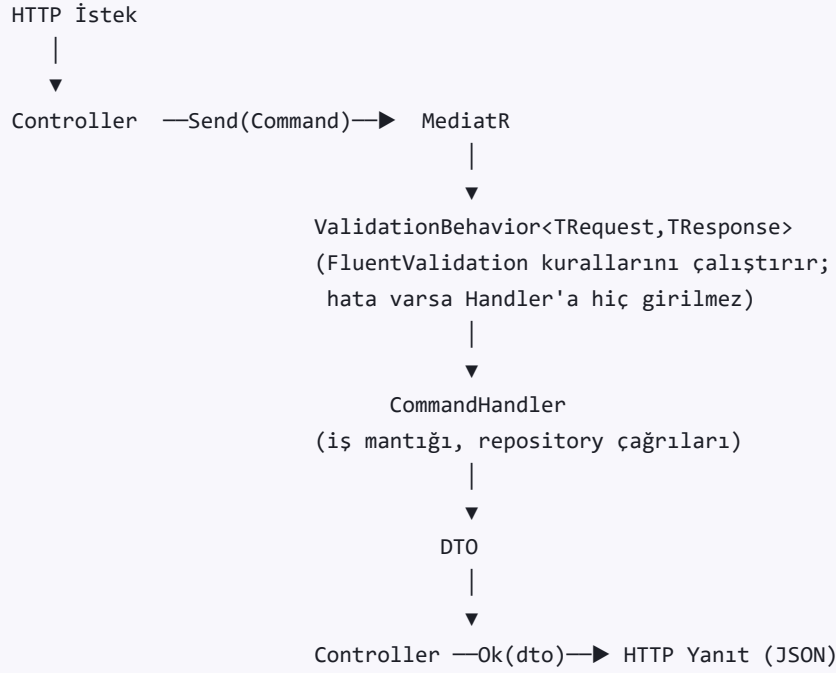
Backend, bağımlılıkların içe doğru aktığı klasik **Onion Architecture** ile yapılandırılmıştır. Her katman yalnızca kendinden içeride olanı bilir; dış katmanlar (Infrastructure, Presentation) hiçbir zaman iç katmanlara (Domain) sızmaz.

```
backend/  
├── Core/  
│   ├── CineSeat.Domain/           (en iç katman: Entity, Enum)  
│   └── CineSeat.Application/       (CQRS: Command/Query, Handler, DTO,  
│                                   Validator, Repository arayüzleri)  
├── Infrastructure/  
│   ├── CineSeat.Infrastructure/    (JWT üretimi, parola hash'leme)  
│   └── CineSeat.Persistence/        (EF Core, PostgreSQL, Migration'lar,  
│                                   Repository implementasyonları)  
└── Presentation/  
    └── CineSeat.WebAPI/             (Controller'lar, Middleware,  
                                    Program.cs – HTTP'nin tek giriş noktası)
```

Neden bu ayrım? `CineSeat.Application`, veritabanının PostgreSQL olduğunu ya da HTTP ile mi yoksa başka bir taşıyıcıyla mı çağrıldığını bilmez — yalnızca arayüzler (`IUserReadRepository`, `ITokenService` gibi) üzerinden çalışır. Somut implementasyonlar (EF Core sorguları, JWT kütüphanesi) dışarıdaki katmanlarda yaşar ve `Program.cs` 'te bağımlılık enjeksiyonu (DI) ile içeri "takılır". Bu sayede iş mantığı test edilirken gerçek bir veritabanına ihtiyaç duyulmaz.

2.2 CQRS ve MediatR Boru Hattı

Uygulama katmanı, **CQRS** (Command Query Responsibility Segregation) deseniyle yazılmıştır: her HTTP isteği bir `Command` (veri değiştiren: giriş, kitleme, rezervasyon oluşturma) ya da bir `Query` 'e (veri okuyan: film listesi, seans listesi) karşılık gelir. Bu nesneler, **MediatR** kütüphanesi aracılığıyla ilgili `Handler` 'a yönlendirilir; controller'lar iş mantığını hiç bilmez, yalnızca isteği MediatR'a "gönderir" (`_mediator.Send(command)`).



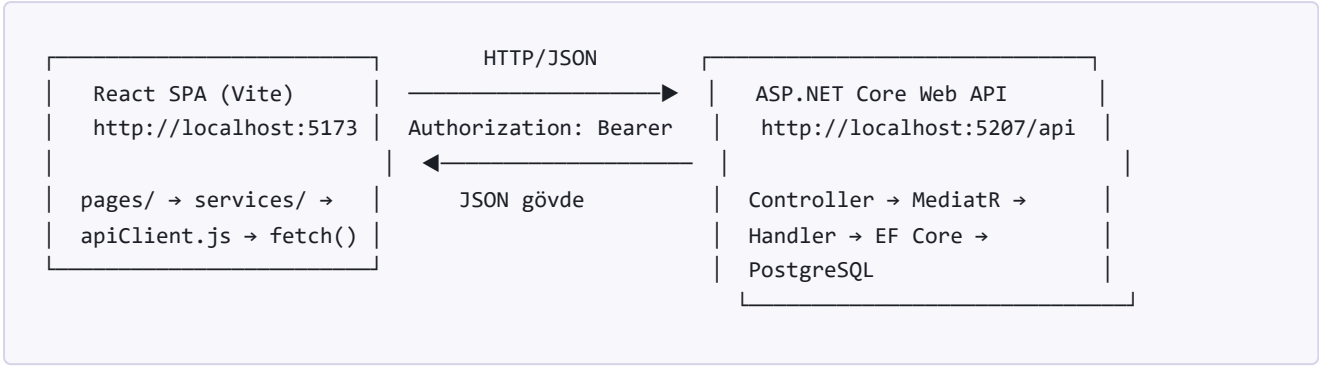
Bu boru hattının en önemli kazancı: **doğrulama kodu hiçbir handler'ın içinde tekrar yazılmaz**. Yeni bir `Command` için bir `Validator` sınıfı eklemek yeterlidir — `ValidationBehavior` onu otomatik olarak bulur ve devreye sokar (bkz. Bölüm 8).

2.3 Frontend Katmanları

```
frontend/src/
├── pages/           (rota başına bir sayfa bileşeni – ör. PaymentPage.jsx)
├── components/     (yeniden kullanılan UI parçaları – ör. Stepper, Header)
├── context/        (AuthProvider – oturum durumu, React Context ile global)
├── hooks/          (useCart, useCountdown, useTabs gibi paylaşılan davranışlar)
└── services/       (★ BU RAPORUN KONUSU: backend'e giden HER istek buradan geçer)
```

Frontend tarafında backend ile konuşan tek katman `frontend/src/services/` dizinidir. Sayfa bileşenleri (React Query `useQuery` / `useMutation` ile) doğrudan bu servis fonksiyonlarını çağırır; hiçbir bileşen `fetch()` 'i doğrudan kullanmaz. Bu, backend'in URL yapısı değişse bile yalnızca ilgili servis dosyasının güncellenmesi gerektiği anlamına gelir.

2.4 İki Uygulama Arasındaki Genel Şema



İki uygulama **tamamen ayrı süreçlerdir** ve yalnızca HTTP üzerinden, JSON gövdeli isteklerle konuşur. Bu ayırım, aşağıdaki üç teknik problemi doğurur ve bu raporun geri kalanı bu üçünün nasıl çözüldüğünü anlatır: **(1) Tarayıcı güvenliği (CORS)**, **(2) Kimlik doğrulama (JWT)**, **(3) Veri sözleşmesi (DTO/Contract)**.

3. Ağ Katmanı: Portlar, Ortam Değişkenleri, CORS

Frontend geliştirme sunucusu (Vite) varsayılan olarak `5173` portunda, backend ise `5207` portunda (bkz. `launchSettings.json`) çalışır. Farklı port = farklı **origin** demektir ve tarayıcılar, güvenlik gereği, bir origin'den başka bir origin'e yapılan `fetch()` isteklerini varsayılan olarak engeller (Same-Origin Policy). Bu engeli açıkça kaldırmak için backend'in **CORS** (Cross-Origin Resource Sharing) politikası tanımlaması gerekir.

`backend/Presentation/CineSeat.WebAPI/Program.cs`

```
const string FrontendCorsPolicy = "FrontendCorsPolicy";
builder.Services.AddCors(options =>
{
    options.AddPolicy(FrontendCorsPolicy, policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://127.0.0.1:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
// ...
// CORS, kimlik doğrulamadan ÖNCE gelmeli – tarayıcı asıl isteği göndermeden
// önce bir "preflight" (OPTIONS) isteği atar, bu istek hiç token taşımaz.
app.UseCors(FrontendCorsPolicy);
app.UseAuthentication();
app.UseAuthorization();
```

Neden middleware sırası önemli? Tarayıcı, `Authorization` header'ı taşıyan "asıl" isteği göndermeden önce header'sız bir `OPTIONS` ön-uçuş (preflight) isteği gönderir. Eğer `UseCors`, `UseAuthentication` 'dan sonra çağrılıyorsa, bu token'sız preflight isteği kimlik doğrulama katmanına takılıp **401** ile reddedilir, tarayıcı da bunu bir CORS hatası olarak yorumlardı — hata mesajı yanıtıcı olurdu.

Frontend tarafında backend'in adresi tek bir yerde, ortam değişkeni olarak tanımlıdır:

`frontend/src/services/apiClient.js`

```
const API_BASE_URL =
import.meta.env.VITE_API_BASE_URL || "http://localhost:5207/api";
```

Bu satır sayesinde geliştirme, test ve üretim ortamları arasında geçiş yapmak için tek bir `.env` dosyasındaki `VITE_API_BASE_URL` değeri değiştirilir — kod hiçbir yerde elle güncellenmez.

Ortam	Frontend adresi	Backend adresi	Kaynak
Geliřtirme (dev)	<code>http://localhost:5173</code>	<code>http://localhost:5207/api</code>	Vite varsayılanı / <code>launchSettings.json</code>
Üretim (prod)	Statik dosya sunucusu / CDN	<code>VITE_API_BASE_URL</code> ile geçersiz kılınır	<code>frontend/.env.production</code>

4. Kimlik Doğrulama (JWT) — Uçtan Uca Akış

Sistem, oturum durumunu sunucuda saklamayan (stateless) bir **JWT (JSON Web Token, HS256 imzalı)** modeli kullanır. Akış şu şekilde işler:

```
1) POST /api/auth/login { usernameOrEmail, password }
   |
   ▼ (LoginCommandHandler: parola hash doğrulama)
2) Yanıt: { token, expiresAt, user: {...} }
   |
   ▼ (AuthProvider.jsx: sessionStorage'a yazılır)
3) sessionStorage["cineseat_user"] = { ...user, token }
   |
   ▼ (sonraki HER istekte apiClient.js token'ı okur)
4) Her istek → Header: Authorization: Bearer <token>
   |
   ▼ (backend: JwtBearer middleware doğrular)
5a) Geçerli → [Authorize] ile işaretli endpoint'e giriş izni
5b) Geçersiz/süresi dolmuş → 401 → apiClient.js oturumu düşürür
```

4.1 Backend: Token Üretimi

backend/Infrastructure/CineSeat.Infrastructure/Security/JwtTokenService.cs

```
public AccessTokenDto CreateAccessToken(
    long userId, string username, string email,
    string roleName, IEnumerable<string> permissions)
{
    var claims = new List<Claim>
    {
        new(ClaimTypes.NameIdentifier, userId.ToString()), // ICurrentUserService.UserId bunu okur
        new(ClaimTypes.Name, username),
        new(ClaimTypes.Email, email),
        new(ClaimTypes.Role, roleName), // [Authorize(Roles="Admin")] bunu okur
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };
    claims.AddRange(permissions.Select(p => new Claim(PermissionClaimTypes.Permission, p)));

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
    var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer: _settings.Issuer, audience: _settings.Audience,
        claims: claims, notBefore: DateTime.UtcNow,
        expires: DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes),
        signingCredentials: credentials);
    // ...
}
```

Token'a gömülen **claim'ler** (kullanıcı kimliği, rol, izin listesi) sayesinde her istekte veritabanına gidip "bu kullanıcının izinleri neler?" diye sormaya gerek kalmaz — imzalı token zaten bu bilgiyi taşır. `Program.cs` 'te uygulama açılışında anahtar uzunluğu da denetlenir:

```
if (jwtSettings.Key.Length < 32)
    throw new InvalidOperationException("Jwt:Key en az 32 karakter olmalıdır.");
// HS256 anahtarı kısa olursa imza güvenliği çöker; sessizce geçmek yerine
// uygulama açılışta patlasın.
```

4.2 Frontend: Token'ın Saklanması ve Her İstekte Eklenmesi

`frontend/src/services/apiClient.js`

```
function getStoredToken() {
    const raw = sessionStorage.getItem(USER_STORAGE_KEY); // "cineseat_user"
    if (!raw) return null;
    const parsed = JSON.parse(raw);
    return parsed?.token ?? null;
}

async function apiRequest(path, { method = "GET", body, headers } = {}) {
    const token = getStoredToken();
    const response = await fetch(`${API_BASE_URL}${path}`, {
        method,
        headers: {
            ...(body !== undefined ? { "Content-Type": "application/json" } : {}),
            ...(token ? { Authorization: `Bearer ${token}` } : {}),
            ...headers,
        },
        body: body !== undefined ? JSON.stringify(body) : undefined,
    });
    // ...
}
```

Kritik nokta: Token, ayrı bir `token` alanında değil, kullanıcı nesnesinin (`AuthProvider` 'in `sessionStorage` 'a yazdığı) **içine gömülü** tutulur. Bu, `apiClient.js` 'in React'i (hook, context, router) hiç tanımadan — yalnızca `sessionStorage` 'ı okuyarak — token'a erişebilmesini sağlar. Böylece HTTP katmanı ile UI durum yönetimi birbirinden tamamen bağımsız kalır.

4.3 Oturum Düşürme (401 Yakalama)

`frontend/src/services/apiClient.js` & `frontend/src/context/AuthProvider.jsx`

```
// apiClient.js - 401 geldiğinde React'e haber verecek geri çağırım
let onUnauthorized = null;
export function setUnauthorizedHandler(handler) { onUnauthorized = handler; }

// ... apiRequest içinde:
if (response.status === 401 && token && onUnauthorized) {
  onUnauthorized(); // yalnızca ELİMİZDE bir token varken gelen 401 "süresi doldu" demektir
}

// AuthProvider.jsx - mount olurken kendini kaydeder
useEffect(() => {
  setUnauthorizedHandler(() => {
    setIsSessionExpired(true);
    clearSession(); // sessionStorage temizlenir, sepet sıfırlanır
  });
  return () => setUnauthorizedHandler(null);
}, [clearSession]);
```

Neden bu dolaylı (callback) tasarım? `apiClient.js` , servis katmanının en altında, React'ten habersiz saf bir JavaScript modülüdür. Ama token süresi dolduğunda kullanıcıyı çıkış yaptırmak React state'ini (Context) değiştirmeyi gerektirir. `setUnauthorizedHandler` bu iki dünyayı birbirine bağımlı kılmadan köprüler: `apiClient.js` React'i import etmez, yalnızca "birisi beni dinliyorsa haber ver" der.

5. İstek/Yanıt Sözleşmesi (Contract)

5.1 JSON Serileştirme Kuralları

Backend C# tarafında `PascalCase` property adları kullanır (`UserId`, `ShowtimeId`); frontend JavaScript tarafı ise `camelCase` bekler (`userId`, `showtimeId`). Bu dönüşüm ASP.NET Core'un varsayılan `System.Text.Json` serileştiricisi tarafından otomatik yapılır. Ayrıca enum'lar sayı olarak değil, **ad** olarak taşınır:

`backend/Presentation/CineSeat.WebAPI/Program.cs`

```
builder.Services
    .AddControllers()
    .AddJsonOptions(options =>
        options.JsonSerializerOptions.Converters.Add(
            new JsonStringEnumConverter()));
// Enum'lar JSON'da sayı değil ad olarak taşınır ("Adult", "IMAX").
// Sayı taşındığında istemci tarafında ayrı bir eşleme tablosu tutmak
// gerekiyordu; enum'a yeni değer eklendiğinde iki taraf sessizce ayrıştırdı.
```

Örnek: `ScreeningFormat.IMAX` enum değeri, JSON gövdesinde sayısal `2` değil, doğrudan `"IMAX"` string'i olarak gelir/gider — frontend'in `SCREENING_FORMATS` listesindeki `value` alanlarıyla birebir eşleşir (`showtimeService.js`).

5.2 DTO Eşleme Deseni

Backend her `Query / Command` için özel bir **DTO** (Data Transfer Object) döner — hiçbir zaman EF Core entity'si doğrudan JSON'a serileştirilmez (entity'ler ilişkisel referanslar taşır, sonsuz döngüye ve gereksiz veri sızıntısına yol açar). Frontend tarafında ise her servis dosyası, gelen DTO'yu kendi bileşenlerinin beklediği şekle çeviren bir `mapXDto` fonksiyonu içerir:

`frontend/src/services/showtimeService.js`

```
function mapShowtimeDto(dto) {
    return {
        id: dto.id,
        movieId: dto.movieId,
        hallId: dto.hallId,
        startDatetime: dto.startDatetime,
        basePrice: Number(dto.basePrice) || 0,
        format: dto.format,
        hallName: dto.hallName ?? "",
        cinemaName: dto.cinemaName ?? "",
        totalSeats: dto.totalSeats ?? 0,
    };
}
```

Neden bu ekstra katman? Backend'in DTO şekli değişse (bir alan yeniden adlandırılrsa, tipi değişse) bile, bu değişiklik yalnızca ilgili `mapXDto` fonksiyonunda düzeltilir; sayfa bileşenleri hiç etkilenmez. Bu, backend ve frontend'in **bağımsız olarak evrilebilmesini** sağlayan en önemli mimari karardır.

6. Merkezi API İstemcisi: `apiClient.js`

`frontend/src/services/apiClient.js`, tüm servis dosyalarının kullandığı **tek giriş noktasıdır**. Base URL, Authorization header'ı ve hata çevirisi burada merkezi olarak yapılır — hiçbir servis dosyası `fetch` veya token detayını bilmek zorunda kalmaz.

`frontend/src/services/apiClient.js`

```
export const apiClient = {
  get: (path) => apiRequest(path),
  post: (path, body) => apiRequest(path, { method: "POST", body }),
  put: (path, body) => apiRequest(path, { method: "PUT", body }),
  del: (path) => apiRequest(path, { method: "DELETE" }),
};
```

Ağ hatalarını (backend hiç ayakta değilse) da tek noktada, kullanıcının anlayacağı bir mesaja çevirir:

```
const response = await fetch(...).catch(() => {
  throw new ApiError(
    "Sunucuya ulaşılamıyor. Backend çalışıyor mu kontrol edin.",
    { status: 0, code: "NETWORK_ERROR" }
  );
});
```

Bu satır, kimlik bilgileri doğru olsa bile backend kapalıysa kullanıcıya belirsiz bir `TypeError: Failed to fetch` yerine anlamlı bir mesaj gösterilmesini sağlar — geliştirme sırasında en sık karşılaşılan “bağlantı sorunu” senaryosu tam olarak budur.

7. Servis Katmanı: Her Dosyanın Görevi

Frontend, backend ile konuşan mantığı 20 servis dosyasına böler; her dosya tek bir kaynak/özellik alanından sorumludur (Single Responsibility). Aşağıdaki tablo her dosyanın görevini ve çağırdığı temel uç noktaları listeler.

Dosya	Sorumluluk	Örnek Uç Noktalar
apiClient.js	Merkezi HTTP istemcisi, token ekleme, hata çevirisi	(tüm isteklerin geçtiği ortak katman)
authService.js	Giriş/kayıt, backend↔frontend rol eşlemesi	POST /auth/login , /auth/register
movieService.js	Film kataloğu, filtreleme	GET /movies
cinemaService.js	Sinema/şehir/ilçe listeleri	GET /cinemas
showtimeService.js	Seans listeleme (sinema kırılımında), CRUD (admin)	/showtimes/by-cinema/{id}
seatService.js	Koltuk haritası, kilitleme/yenileme/bırakma	/showtimes/{id}/seats , /seatlocks
seatLockStorage.js	Aktif kilitlerin localStorage 'da saklanması (sayfa yenilense de kilit bilgisi kaybolmaz)	— (yalnızca istemci tarafı)
reservationService.js	Rezervasyon (bilet) oluşturma	POST /reservations
campaignService.js	Kampanya listeleme + istemci tarafı indirim ÖN İZLEMESİ	/campaigns/active , /campaigns (admin)
pricing.js	Sepet/kalem toplamı hesaplama (saf fonksiyonlar, backend'e gitmez)	—
paymentAdapter.js	Ödeme simülasyonu (kart no. deseniyle başarı/red/hata üretir)	— (backend'e bağlı değil, demo amaçlı)
commentService.js	Film yorum/puanlama	/comments
favoriteService.js	İzleme listesi (favoriler)	/favorites
profileService.js	Kullanıcı profili görüntüleme/güncelleme	/profile
userService.js	Yönetim: kullanıcı listesi, rol atama	/users , /roles
venueService.js	Yönetim: salon/koltuk düzeni	/halls
locationService.js	Konum bazlı en yakın sinema hesaplama (Haversine, istemci tarafı)	—
adminResource.js	Yönetim CRUD ekranları için jenerik sayfalama yardımcıları (fetchAllPages)	(tüm /admin -benzeri listeleme uçları)
validation.js	Form doğrulama kuralları (istemci tarafı, backend FluentValidation'ı yansıtır)	—

errors.js	Tipli hata sınıfları (<code>ApiError</code> , <code>ValidationError</code> , <code>NotFoundError</code> , <code>ConflictError</code> , <code>ForbiddenError</code>)	—
-----------	---	---

8. CQRS Zinciri — Uçtan Uca Örnek: Giriş (Login)

En küçük parçaya kadar takip edilebilir bir örnek olarak **giriş (login)** isteğinin backend içindeki tam yolculuğu:

8.1 Controller — HTTP'yi MediatR'a Devreder

backend/Presentation/CineSeat.WebAPI/Controllers/AuthController.cs

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IMediator _mediator;

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginCommand command)
        => Ok(await _mediator.Send(command));
}
```

Controller, iş mantığı hakkında **hiçbir şey bilmez**. Yalnızca gelen JSON gövdesini `LoginCommand` 'a otomatik bağlar (model binding) ve MediatR'a gönderir.

8.2 Command — İsteğin Şekli

backend/Core/CineSeat.Application/Features/Auth/Commands/Login/LoginCommand.cs

```
public class LoginCommand : IRequest<AuthResultDto>
{
    public string UsernameOrEmail { get; set; } = "";
    public string Password { get; set; } = "";
}
```

8.3 Validator — ValidationBehavior Tarafından Otomatik Bulunur

.../Login/LoginCommandValidator.cs

```
public class LoginCommandValidator : AbstractValidator<LoginCommand>
{
    public LoginCommandValidator()
    {
        RuleFor(x => x.UsernameOrEmail).NotEmpty().MaxLength(150);
        RuleFor(x => x.Password).NotEmpty().MaxLength(100);
    }
}
```

backend/Core/CineSeat.Application/Common/Behaviors/ValidationBehavior.cs

```
public class ValidationBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
    CancellationToken ct)
    {
        if (!_validators.Any()) return await next(); // bu isteğin validator'ı yoksa doğrudan geç

        var results = await Task.WhenAll(_validators.Select(v => v.ValidateAsync(new
        ValidationContext<TRequest>(request), ct)));
        var failures = results.SelectMany(r => r.Errors).ToList();

        if (failures.Count > 0)
            throw new ValidationException(failures); // Handler'a hiç girilmez; middleware bunu
            400'e çevirir

        return await next();
    }
}
```

8.4 Handler — İş Mantığı

.../Login/LoginCommandHandler.cs

```
public async Task<AuthResultDto> Handle(LoginCommand request, CancellationToken ct)
{
    var identifier = request.UsernameOrEmail.Trim().ToLower();
    var candidate = await _queryExecutor.FirstOrDefaultAsync(
        _userRead.GetWhere(u => u.Email.ToLower() == identifier || u.Username.ToLower() ==
        identifier, tracking: false)
        .Select(u => new { u.Id, u.Name, u.PasswordHash, u.PasswordSalt, RoleName = u.Role.Name,
        ... })), ct);

    // Kullanıcı yok İLE parola yanlış AYNI mesajı döndürür: farklı mesaj
    // vermek saldırganı hangi e-postaların kayıtlı olduğunu sızdırır.
    if (candidate is null || !_passwordHasher.Verify(request.Password, candidate.PasswordHash,
    candidate.PasswordSalt))
        throw new UnauthorizedException("Kullanıcı adı/e-posta veya parola hatalı.");

    var token = _tokenService.CreateAccessToken(candidate.Id, candidate.Username, candidate.Email,
    candidate.RoleName, candidate.Permissions);
    return new AuthResultDto { Token = token.Token, ExpiresAt = token.ExpiresAt, User = new
    AuthUserDto { ... } };
}
```

Güvenlik notu: “Kullanıcı bulunamadı” ile “parola yanlış” durumları **kasıtlı olarak** aynı hata mesajını (`UnauthorizedException`) üretir. Bu, bir saldırganın deneme-yanılma ile sistemde kayıtlı e-posta adreslerini tespit etmesini (user enumeration) engeller — OWASP'ın kimlik doğrulama en iyi pratiklerinden biridir.

8.5 Frontend Tarafı — Yanıtın Karşılanması

frontend/src/services/authService.js

```
login: async (email, password) => {  
  const result = await apiClient.post("/auth/login", { usernameOrEmail: email, password });  
  return mapAuthResult(result); // backend'in { token, user } şeklini AuthProvider'ın beklediği  
    şekle çevirir  
},
```

Bu tek örnek — controller → command → validator → handler → DTO → frontend mapping — projedeki **her** yazma işlemi (kayıt, kilitleme, rezervasyon, yorum ekleme, kampanya oluşturma) için birebir aynı iskeleti izler; yalnızca isimler ve iş kuralları değişir.

9. Hata Yönetimi Zinciri

Backend'de **tek bir try/catch noktası** vardır: `ExceptionHandlerMiddleware`. Hiçbir controller veya handler kendi içinde hata yakalamaz; bunun yerine anlamlı özel exception türleri (`NotFoundException`, `ConflictException`, `UnauthorizedException`, `ValidationException`) fırlatır ve middleware bunları HTTP durum koduna + tutarlı bir JSON gövdesine çevirir.

`backend/Presentation/CineSeat.WebAPI/Middleware/ExceptionHandlerMiddleware.cs`

```
catch (ValidationException ex) { // → 400, { title, status, errors: {alan: [mesaj,...]} }
catch (NotFoundException ex) { // → 404, { title, status, detail }
catch (UnauthorizedException ex) { // → 401, { title, status, detail }
catch (ConflictException ex) { // → 409, { title, status, detail }
catch (Exception ex) { // → 500, Loglanır, gövde kullanıcıya iç detay sızdırmaz
```

Frontend tarafında bu tutarlı gövde, `apiClient.js`'in `toApiError()` fonksiyonunda aynı durum kodlarına göre **tipli hata sınıflarına** çevrilir:

`frontend/src/services/apiClient.js`

```
switch (response.status) {
  case 400: case 422: return new ValidationError(message);
  case 401: return new ApiError(message, { status: 401, code: "UNAUTHORIZED" });
  case 403: return new ForbiddenError(message);
  case 404: return new NotFoundError(message);
  case 409: return new ConflictError(message);
  default: return new ApiError(message, { status: response.status });
}
```

Bu iki uç (backend middleware ↔ frontend `toApiError`) birbirinin **aynasıdır**. Sayfa bileşenleri `err.message` 'i okur ve kullanıcıya gösterir; hangi HTTP durumunun hangi Türkçe mesaja karşılık geldiğini hiç bilmek zorunda kalmazlar.

10. Sayfalama Sözleşmesi

Listeleme uçları (`/movies`, `/comments`, `/showtimes/by-cinema/{id}` vb.) sayfalı (`PagedResult<T>`) yanıt döner ve sunucu tarafında **sayfa başına en fazla 100 kayıt** ile sınırlıdır (FluentValidation ile doğrulanır). Frontend'in jenerik yardımcı fonksiyonu bu sınırı otomatik gözetir:

frontend/src/services/adminResource.js

```
export const MAX_PAGE_SIZE = 100;

export async function fetchAllPages(basePath, params = {}) {
  // Çağırılan belirli bir sayfa istediye ona karışılmaz.
  // Aksi halde MAX_PAGE_SIZE'la sayfa sayfa dolaşır tüm kayıtları toplar.
  const collected = [];
  query.set("pageSize", String(MAX_PAGE_SIZE));
  for (let pageNumber = 1; pageNumber <= MAX_PAGES; pageNumber += 1) {
    query.set("pageNumber", String(pageNumber));
    const result = await request();
    // ...sayfaları birleştirip totalPages'e ulaşınca durur
  }
}
```

Gerçek vaka (bu entegrasyonda yakalanan bir hata): Yönetim panelindeki seans listesi eskiden `pageSize=200` gönderiyordu; backend'in doğrulayıcısı bunu reddedip **400** döndürüyor, ekran "Seanslar yükleniyor..." durumunda sonsuza kadar kalıyordu. Çözüm sınırı yükseltmek değil, **sayfaları dolaşmak** oldu — çünkü bir sinemanın seansları rahatlıkla 100'ü aşabiliyor. Bu, istemci ile sunucunun aynı sözleşmeye (100 sınırı) sahip olması gerektiğinin somut bir örneğidir.

Alan	Tip	Açıklama
<code>items</code>	Array	O sayfadaki kayıtlar
<code>totalCount</code>	number	Toplam kayıt sayısı (tüm sayfalar)
<code>page</code>	number	Geçerli sayfa numarası
<code>pageSize</code>	number	Sayfa başına kayıt (≤ 100)
<code>totalPages</code>	number	<code>ceil(totalCount / pageSize)</code>

11. Özel Akış: Koltuk Kilitleme (Concurrency)

Bilet satın alma akışının en kritik teknik problemi budur: **iki farklı kullanıcı aynı koltuğu aynı anda seçerse ne olur?** Çözüm, backend'de veritabanı seviyesinde bir **benzersizlik kısıtı** ((ShowtimeId, SeatId) üzerinde UNIQUE) ve bunu yöneten bir "devralma" (takeover) mantığıdır.

11.1 Frontend — Ödeme Sayfasına Girişte Kilit Alma

frontend/src/pages/PaymentPage.jsx

```
// Koltukları kilitler. Sepet ödeme sırasında değişmediği için bu etki
// yalnızca girişte bir kez çalışır.
useEffect(() => {
  async function acquireLocks() {
    const acquired = [];
    try {
      for (const item of state.items) {
        const locks = await seatService.lockSeats({
          showtimeId: item.sessionId,
          seatIds: item.seats.map((seat) => seat.seatId),
        });
        acquired.push(...locks);
      }
    } catch (error) {
      await seatService.releaseLocks(acquired.map((lock) => lock.id)); // kısmi başarıyı geri al
      setLockError(error.message || "Seçtiğiniz koltuklardan biri artık müsait değil.");
      return;
    }
    // ...
  }
}, []);
```

11.2 Backend — Devralma (Takeover) Mantığı

backend/Core/CineSeat.Application/Features/SeatLocks/Commands/LockSeat/LockSeatCommandHandler.cs

```

var existingLock = await _lockRead.GetSingleAsync(sl => sl.ShowtimeId == ... && sl.SeatId == ...,
tracking: true, ct);

if (existingLock is null)
{
    var newLock = new SeatLock { ShowtimeId = ..., SeatId = ..., UserId = userId, LockExpiresAt =
now.AddMinutes(...) };
    await _lockWrite.AddAsync(newLock, ct);
    try { await _lockWrite.SaveAsync(ct); return ToDto(newLock); }
    catch (ConflictException)
    {
        // YARIŞ: okuma ile yazma arasında BAŞKA bir istek aynı (ShowtimeId, SeatId)
        // satırını ekledi ve benzersiz izin bizimkini reddetti. Bu her zaman
        // gerçek bir çakışma DEĞİL: aynı kullanıcı aynı koltuğu eş zamanlı iki
        // kez isteyebiliyor (React'in geliştirme modunda efekti iki kez
        // çalıştırması bunu düzenli olarak tetikliyordu).
        _lockWrite.Detach(newLock);
        existingLock = await _lockRead.GetSingleAsync(...);
        if (existingLock is null) throw;
    }
}

var isExpired = existingLock.LockExpiresAt < now;
var isSameUser = existingLock.UserId == userId;

if (!isExpired && !isSameUser)
    throw new ConflictException("Bu koltuk şu anda başka bir kullanıcı tarafından kilitli.");

// Süresi dolmuş ya da aynı kullanıcı → satırı devral / yenile.
existingLock.UserId = userId;
existingLock.LockExpiresAt = now.AddMinutes(...);
_lockWrite.Update(existingLock);
await _lockWrite.SaveAsync(ct);

```

Neden bu tasarım sağlam? Kilit satırı benzersizlik kısıtıyla korunur, bu yüzden iki eşzamanlı istek aynı satırı **aynı anda** oluşturamaz — veritabanı bunlardan birini reddeder. Kod, bu reddi bir hata olarak kullanıcıya yansıtmak yerine yakalar, satırı yeniden okur ve “süresi dolmuş mu / benim mi?” sorusuna göre devam eder. Bu, dağıtık sistemlerde **iyimser eşzamanlılık kontrolü** (optimistic concurrency) desenine örnektir.

11.3 Bırakma — Kasıtlı İdempotentlik

Kilit bırakma (`DELETE /seatlocks/{id}`) işlemi, kilit zaten yoksa veya başka birine geçmişse dahi hep **204** döner (hata fırlatmaz). Frontend'deki `releaseLocks` bir temizlik işidir; kilidin süresi zaten dolmuşsa gösterilecek bir hata yoktur:

```

// Kilitleri bırakır. Temizlik işi olduğu için tek tek hatalar yutulur.
async function releaseLocks(lockIds = []) {
    await Promise.all(lockIds.map((lockId) => apiClient.del(`/seatlocks/${lockId}`).catch(() =>
null)));
}

```


12. Özel Akış: Kampanya/İndirim Önizlemesi

Sepet ve ödeme sayfalarında görünen indirim tutarı, güvenlik açısından öğretici bir mimari kararı örnekler: **istemci hiçbir zaman bağlayıcı bir tutar hesaplayıp sunucuya göndermez.**

frontend/src/services/campaignService.js

```
/**
 * Kampanyalar artık backend'den geliyor. İndirimin BAĞLAYICI hesabı da
 * backend'de yapılır (rezervasyon oluşturulurken); buradaki hesap yalnızca
 * sepette ÖN İZLEME içindir – istemciden gelen tutar sunucuya gönderilmez,
 * yalnızca seçilen kampanyanın `id`'si gönderilir.
 */
```

Frontend, backend'in kural setini (yüzde/sabit tutar, minimum sepet tutarı, sadece üyelere özel) birebir yansıtan saf fonksiyonlarla **aynı sonucu tahmin eder** — ama nihai doğrulama ve tahsilat her zaman sunucuda, `ReservationCommandHandler` tarafından yapılır. Kullanıcı tarayıcı konsolundan istemci tarafı hesabı manipüle etse dahi, sunucu kendi hesabını yeniden yapar ve göndereni değil kendi sonucunu esas alır.

Kalem bazında hesap: Backend her seans için ayrı bir rezervasyon oluşturur ve kampanya koşullarını (özellikle `minCartTotal`) o rezervasyonun kendi ara toplamına göre doğrular. Bu nedenle istemci de indirim ön izlemesini **sepetin tamamına değil, her kaleme (seansa) ayrı ayrı** uygulamak zorundadır (`planCampaignsPerItem`) — aksi halde ekranda görünen ön izleme ile ödeme anında sunucunun uyguladığı tutar birbirini tutmaz.

13. Dosya–İşlev Referans Tablosu

Bu tablo, entegrasyonda rol oynayan en önemli dosyaları katman ve görevleriyle birlikte tek bakışta özetler.

Katman	Dosya	Görev
Backend — Presentation	Program.cs	DI kurulumu, CORS, JWT middleware, pipeline sırası
	Middleware/ExceptionHandlingMiddleware.cs	Tek merkezi hata yakalama → tutarlı JSON gövdesi
	Controllers/*.cs	HTTP → MediatR köprüsü, iş mantığı içermez
	Services/BearerSecuritySchemeTransformer.cs	Swagger/OpenAPI'de Bearer şeması tanımı
Backend — Application	Features/*/Commands Queries/*	Her iş eylemi için Command/Query + Handler + Validator üçlüsü
	Common/Behaviors/ValidationBehavior.cs	MediatR boru hattında otomatik doğrulama
	Common/Exceptions/*.cs	Katmanlar arası anlamlı hata türleri
	Repositories/*.cs (arayüzler)	Application'ın veri erişimini soyutlaması
Backend — Infrastructure	Security/JwtTokenService.cs	JWT üretimi (HS256)
	Persistence/Repositories/*.cs	EF Core ile somut veri erişimi
Frontend — Servis Katmanı	services/apiClient.js	Tüm isteklerin geçtiği tek HTTP katmanı
	services/*Service.js (20 dosya)	Kaynak bazlı istek + DTO→model eşleme
	services/errors.js	Backend hata koduna karşılık gelen tipli sınıflar
Frontend — Durum/Bağlam	context/AuthProvider.jsx	Oturum durumu, 401 dinleyicisi kaydı
	hooks/useCart.js	Sepet durumu (istemci tarafı, backend'e yalnızca checkout'ta gider)
	hooks/useCountdown.js	Koltuk kilidi geri sayımı (UI, backend'in lockExpiresAt 'ine göre)

14. Canlı Doğrulama Notları

Bu rapor yalnızca statik kod incelemesine dayanmaz; entegrasyonun gerçekten çalıştığı, backend gerçek bir PostgreSQL veritabanına bağlı şekilde ayağa kaldırılarak **canlı olarak** doğrulanmıştır. Doğrulanmış akışlar:

Akış	Adımlar	Sonuç
CORS ön-uçuş	<code>OPTIONS /api/movies ,</code> <code>Origin: http://localhost:5173</code>	204 — beklenen
Kayıt → Giriş → Profil → Favoriler	<code>POST /auth/register</code> → <code>POST /auth/login</code> → <code>GET /profile</code> → <code>GET /favorites</code> (token ile)	4/4 istek 200
Yönetici uçları	<code>GET /users</code> , <code>GET /roles</code> (admin token ile)	200 — yetkilendirme doğru çalışıyor
Sayfalama sınırı	<code>pageSize=200</code> ile <code>/movies</code> , <code>/comments</code> , <code>/showtimes/by-cinema/1</code>	3/3 istek 400 — doğrulayıcı doğru reddediyor
Koltuk kilidi yaşam döngüsü	kilitle → aynı koltuğu tekrar kilitle (devral) → yenile → bırak → tekrar bırak	5/5 adım beklenen kod (200/200/200/204/204)

Sonuç: entegrasyonun temel taşlarında (CORS, JWT, sayfalama sözleşmesi, koltuk kilidi eşzamanlılığı) canlı, çalışan bir sistemde doğrulanmış bir sorun bulunmamaktadır.

15. Sonuç

CineSeat'te frontend ve backend, birbirinden tamamen bağımsız iki uygulama olarak geliştirilmiş; aralarındaki bağlantı üç temel sözleşme üzerine kurulmuştur:

1. **Ağ/güvenlik sözleşmesi:** CORS politikası ile hangi origin'in istek atabileceği açıkça tanımlanmıştır.
2. **Kimlik sözleşmesi:** Stateless JWT, her istekte `Authorization: Bearer` header'ı ile taşınır; oturum sunucuda değil token'ın kendisinde yaşar.
3. **Veri sözleşmesi:** Backend DTO'ları ile frontend'in `mapXDto` fonksiyonları arasındaki eşleme, iki tarafın birbirinden bağımsız evrilmesine izin verir; hata gövdeleri ve sayfalama şekli tutarlı ve öngörülebilirdir.

Bu üç sözleşmenin her biri kod üzerinde somut, isimlendirilebilir bir karşılığa sahiptir (`Program.cs` 'teki `AddCors` , `ApiClient.js` 'teki `getStoredToken` , her serviste tekrar eden `mapXDto` deseni) ve bu rapor boyunca gösterildiği gibi, en kritik iş akışında (koltuk kilitleme) dahi eşzamanlılık sorunları bilinçli ve test edilmiş bir tasarımla çözülmüştür.